DevOps Laboratory

Assignment 1

Git Repository and Branch Management

Student Name	YUVRAJ SHEMBALE
Roll No / PRN	123B1D045
Date of Submission	

1. Aim

To understand Git's 3-tree architecture, practice core Git commands, and demonstrate branch creation, merging, and the difference between git reset and git revert.

2. Introduction

Git is a distributed version control system (DVCS) used to track changes in source code and support collaborative software development. Unlike centralized version control systems, every developer's working copy of a Git repository contains the full history of the project, enabling offline commits, branching, and merging.

Git allows developers to maintain different versions of a project, work independently on separate branches, and safely combine changes later. Central to Git's design is the 3-Tree Architecture consisting of the Working Directory, the Staging Area (Index), and the Repository (.git directory) — understanding how files move between these three areas is essential for effective version control and forms the foundation of most DevOps workflows.

3. Technologies / Tools Used

Technology	Purpose / Role
Git	Distributed version control system used to track source code history
GitHub	Remote/cloud hosting platform for Git repositories, used for collaboration
Bash / Terminal	Command-line interface used to execute Git commands
Text Editor (VS Code)	Used to create and modify project files

4. Project / Repository Structure

```
git-branch-demo/
■■■ .git/                # Git metadata (created by git init)
■■■ .gitignore
■■■ README.md
■■■ project.txt          # sample project file
■■■ docs/
    ■■■ notes.txt
```

5. Procedure

1. Initialize a local Git repository using git init, or clone an existing repository from GitHub using git clone .

2. Create a project file (e.g., project.txt) and record the initial change with git add and git commit.

3. Create a new feature branch using git branch feature-branch followed by git checkout feature-branch (or git switch -c feature-branch).

4. Modify the project file on the feature branch and commit the changes with a meaningful commit message.

5. Merge the feature branch into dev using git merge feature-branch and resolve any merge conflicts if they occur.

6. Push the merged changes to the remote GitHub repository using git push origin dev.

7. Use git log --oneline --graph and git show to examine repository history in detail.

8. Demonstrate git reset (moving the branch pointer, optionally discarding changes) versus git revert (creating a new commit that undoes an earlier commit) on a shared branch.

6. Code / Configuration Snippets

6.1 Repository Initialization and First Commit

```
$ git init
$ echo "# Git Branch Demo" > README.md
$ git add README.md
$ git commit -m "Initial commit: add README"
```

6.2 Creating and Switching Branches

```
$ git branch feature-login
$ git switch feature-login
# or in one step:
$ git checkout -b feature-login
```

6.3 Making Changes and Committing on the Feature Branch

```
$ echo "Login module added" >> project.txt
$ git add project.txt
$ git commit -m "Add login module"
```

6.4 Merging the Feature Branch into Main

```
$ git switch main
$ echo "Update from main" >> project.txt
$ git commit -am "Update project.txt on main"
$ git merge feature-login
$ git push origin main
```

6.5 Inspecting History

```
$ git log --oneline --graph --all
$ git show <commit-hash>
```

6.6 git reset vs git revert

```
# git reset: moves HEAD/branch pointer, can discard commits (local/unshared history)
$ git reset --hard HEAD~1

# git revert: creates a NEW commit that undoes changes (safe for shared branches)
$ git revert <commit-hash>
```

7. Output / Screenshots

The following screenshots demonstrate successful execution of each major step. Replace each placeholder with the corresponding screenshot captured during the practical.

```
PS D:\Devops Lab\Lab Assignments\Assignment 1\git-branches> git init
Initialized empty Git repository in D:/Devops Lab/Lab Assignments/Assignment 1/git-branches/.git/
PS D:\Devops Lab\Lab Assignments\Assignment 1\git-branches>
```

Fig 1: Output of git init / git clone showing repository initialization

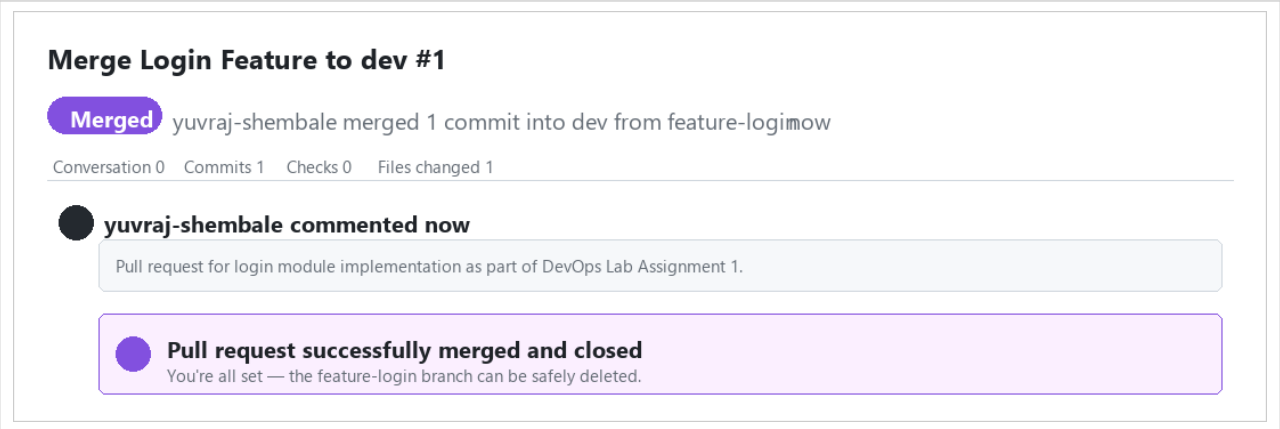


Fig 2: Successful merge of feature branch into dev

```
PS D:\Devops Lab\Lab Assignments\Assignment 1\git-branches> git log --oneline --graph --all
*   03f0c27 (HEAD -> dev, origin/dev) Merge pull request #1 from yuvraj-shembale/feature-login
|\
| * f05064f (origin/feature-login, feature-login) Add login module
|/
* 07ff24e (origin/main, origin/HEAD, main) Initial commit: add README and docs
PS D:\Devops Lab\Lab Assignments\Assignment 1\git-branches>
```

Fig 3: git log --oneline --graph showing commit history

```
PS D:\Devops Lab\Lab Assignments\Assignment 1\git-branches> git revert -m 1 03f0c27
[dev 85150fc] Revert "Merge pull request #1 from yuvraj-shembale/feature-login"
 1 file changed, 0 insertions(+), 0 deletions(-)
 delete mode 100644 project.txt
PS D:\Devops Lab\Lab Assignments\Assignment 1\git-branches> git reset --hard HEAD~1
HEAD is now at 03f0c27 Merge pull request #1 from yuvraj-shembale/feature-login
PS D:\Devops Lab\Lab Assignments\Assignment 1\git-branches>
```

Fig 4: Comparison of repository state after git reset vs git revert

8. Best Practices

- Use meaningful, descriptive commit messages that explain the 'why', not just the 'what'.
- Create separate branches for new features, fixes, or experiments instead of committing directly to main.
- Pull the latest changes (git pull) before starting new work to avoid unnecessary conflicts.
- Avoid committing unnecessary files; use a .gitignore file for build artifacts, logs, and secrets.
- Prefer git revert over git reset on shared/public branches, since revert preserves history safely.

9. Conclusion

Git provides the foundation for collaborative software development by maintaining a complete, distributed history of source code changes. Through this assignment, the working directory, staging area, and repository were used in practice, and a feature branch was created, modified, and merged back into the main branch.

The distinction between git reset and git revert was also demonstrated, highlighting why revert is the safer choice for shared branches. Overall, this exercise reinforced how Git commands, repository structure, and branch management form an essential foundation for implementing DevOps practices.